# International Collegiate Programming Contest

# 2025

Gran Premio de México 2025 -Tercera Fecha

*August 23rd, 2025*

## Contest Session

*This problem set contains 12 problems; pages are numbered from 1 to 15.*

# General information

Unless otherwise stated, the following conditions hold for all problems.

## Program name

1. Your solution must be called *codename*`.c`, *codename*`.cpp`, *codename*`.java`, *codename*`.kt`, *codename*`.py3`, where *codename* is the capital letter which identifies the problem.

## Input

1. The input must be read from standard input.
2. The input consists of a single test case, which is described using a number of lines that depends on the problem. No extra data appear in the input.
3. When a line of data contains several values, they are separated by *single* spaces. No other spaces appear in the input. There are no empty lines.
4. The English alphabet is used. There are no letters with tildes, accents, diaereses or other diacritical marks (ñ, Ã, é, Ì, ô, Ü, ç, etcetera).
5. Every line, including the last one, has the usual end-of-line mark.

## Output

1. The output must be written to standard output.
2. The result of the test case must appear in the output using a number of lines that depends on the problem. No extra data should appear in the output.
3. When a line of results contains several values, they must be separated by *single* spaces. No other spaces should appear in the output. There should be no empty lines.
4. The English alphabet must be used. There should be no letters with tildes, accents, diaereses or other diacritical marks (ñ, Ã, é, Ì, ô, Ü, ç, etcetera).
5. Every line, including the last one, must have the usual end-of-line mark.
6. To output real numbers, round them to the closest rational with the required number of digits after the decimal point. Test case is such that there are no ties when rounding as specified.

# Problem A – Archmage's Crystals

In the high halls of Numeria, an archmage tends to $n$ magic crystals with powers $a_1, a_2, \ldots, a_n$. With a fusion spell, he may pick any two crystals with powers $x$ and $y$, remove them, and create a single crystal of power $x + y$. Each spell reduces the number of crystals by 1.

The archmage seeks harmony: he wants the average power of the crystals to become an integer. What is the minimum number of spells he must cast?

## Input

The first line contains an integer $n$ ($1 \leq n \leq 10^6$).

The second line contains $n$ integers $a_1, a_2, \ldots, a_n$ ($1 \leq a_i \leq 10^3$).

## Output

Print a single integer: the minimum number of spells.

| **Sample input 1** | **Sample output 1** |
|---|---|
| 4<br>5 2 3 5 | 1 |

| **Sample input 2** | **Sample output 2** |
|---|---|
| 5<br>11 12 13 14 15 | 0 |

| **Sample input 3** | **Sample output 3** |
|---|---|
| 4<br>1 5 3 7 | 0 |

# Problem B – Bytelandia's stones

In the mystical Academy of Bytelandia, an archmage is studying a line of $n$ enchanted stones, each with a power value $a_1, a_2, \ldots, a_n$.

A query spell is defined as follows: for two indices $L$ and $R$ ($1 \leq L \leq R \leq n$), the spell reveals the total power of the stones from position $L$ to $R$:

$$S(L, R) = \sum_{i=L}^{R} a_i$$

The oracle wonders: what is the total sum of the answers of all possible queries?

Formally, compute:

$$\left( \sum_{L=1}^{N} \sum_{R=L}^{N} S(L, R) \right)$$

## Input

The first line contains an integer $n$ ($1 \leq n \leq 10^5$) — the number of stones.

The second line contains $n$ integers $a_1, a_2, \ldots, a_n$ ($1 \leq a_i \leq 10^3$) — the values of the stones.

## Output

Print a single integer: the total sum of all queries.

| **Sample input 1** | **Sample output 1** |
|---|---|
| 3<br>1 2 3 | 20 |

| **Sample input 2** | **Sample output 2** |
|---|---|
| 5<br>3 4 7 1 3 | 133 |

| **Sample input 3** | **Sample output 3** |
|---|---|
| 3<br>1 10 100 | 343 |

# Problem C – Colorful Polygon

The *Legendary Huron* owns a long, narrow box filled with magnetic sticks. Each stick has a color. Colors are represented by integers from 1 to $10^6$.

The box can only store sticks in a single line, from left to right. Initially, the box contains $n$ sticks. You are given an array $A$ of length $n$, where $A_i$ is the color of the $i$-th stick from left to right.

Over time, the Legendary Huron will perform $Q_1 + Q_2$ operations on his box. There are four possible operations:

1. Buy a bag of $k$ sticks of color $c$ and insert them to the **left end** of the box.
2. Buy a bag of $k$ sticks of color $c$ and insert them to the **right end** of the box.
3. Remove the **leftmost** $k$ sticks from the box and give them to his cousin, Huroncito.
4. Remove the **rightmost** $k$ sticks from the box and give them to Huroncito.

After each operation, the Legendary Huron will take the sticks from his box to play with them and attempt to build a **colorful polygon** using the sticks he currently possesses. A polygon is *colorful* if it satisfies the following conditions:

- It must be simple with no self-intersections and have a positive area.
- Each side of the polygon must consist of sticks of the same color.
- No two sides may share the same color.
- The length of a side equals the number of sticks used for that side.

For example, if the box has the following sticks $[4, 3, 1, 1, 2, 3, 3, 4, 4]$, where different integers represent different colors, a valid colorful polygon could be a triangle. One side may consist of two sticks of color 1, another side of three sticks of color 3, and the last side of two sticks of color 4. The total perimeter in this case is 7. Note that it is not necessary to use all colors or all sticks of a color.

Each time he plays with the sticks, the Legendary Huron asks for the maximum perimeter of a **colorful polygon** that can be constructed. If no colorful polygon can be formed, the result is 0. After playing with the sticks, they are stored in the same order as before in the box.

The first $Q_1$ operations are given explicitly in the input. The last $Q_2$ operations, however, are not given directly. Instead, they are generated **using a random generator described in the Input section**. **Read the Output section to see how to print the answer for the last $Q_2$ operations**.

## Input

The first line contains three integers $n$, $Q_1$ and $Q_2$ ($1 \le n, Q_1 \le 3 \cdot 10^5$ and $1 \le Q_2 \le 5 \cdot 10^7$).

The second line contains $n$ integers $A_1, A_2, \ldots, A_n$ ($1 \le A_i \le 10^6$) — the initial colors of the sticks in the box.

The next $Q_1$ lines describe the first sequence of operations:

- If the operation type is 1 or 2, the line contains three integers $t$, $k$, and $c$ ($t \in \{1, 2\}$, $1 \le k \le 10^9$, and $1 \le c \le 10^6$).
- If the operation type is 3 or 4, the line contains two integers $t$ and $k$ ($t \in \{3, 4\}$ and $0 \le k \le \min(10^9, sz)$, where $sz$ is the current amount of sticks in the box at the moment of the operation).

After these lines, the input specifies the random generator for the second sequence of operations. The first line of the random generator specification contains two integers $a$ and $d$ ($0 \le a, d < 10^9 + 7$) — the parameters of the generator described below.

The next line contains $X_0$ ($0 \le X_0 < 10^9 + 7$) — the initial seed.

The next line contains an integer $p$ ($1 \le p \le 10^6$) — the size of an array used to generate the last $Q_2$ operations as described below.

The last line in the input contains $p$ integers $B_1, B_2, \ldots B_p$ ($1 \le B_i \le 10^6$) — an array used to generate the last $Q_2$ operations as described below.

The random values used to generate the last $Q_2$ operations are produced by the following linear congruential generator (LCG):

$$X_{i+1} = (a \cdot X_i + d)\%(10^9 + 7)$$

The generator is initialized with $X_0$. Each call to $next()$ returns the next value $X_i$, starting with $X_0$.

Each of the last $Q_2$ operations is then generated as follows:

1. The type is $(next()\%4) + 1$.
2. If the type is 1 or 2:
   - $k = next() + 1$.
   - $c = B[next()\%p) + 1]$.
3. If the type is 3 or 4: $k = next()\%(sz + 1)$, where $sz$ is the current amount of sticks in the box at the moment of the operation.

**You can see an example of the generation of the last $Q_2$ operations in the notes.**

## Output

Print two lines.

In the first line, print $Q_1$ space-separated integers — the answer for the first $Q_1$ operations.

In the second line, print a single integer:

$$\bigoplus_{i=1}^{Q_2} \left(i \cdot res_i \% 998244353\right)$$

where $res_i$ is the result of the $i$-th of the last $Q_2$ operations, and $\oplus$ denotes the bitwise XOR.

**Note that only $i \cdot res_i$ is taken modulo 998244353, not the XOR sum.**

**Note that this modulo is different from the modulo in the generator.**

| **Sample input 1** | **Sample output 1** |
|---|---|
| 5 4 3<br>1 1 2 3 3<br>4 1<br>3 1<br>2 2 4<br>3 2<br>2 3<br>7<br>4<br>3 1 2 4 | 3 3 5 0<br>15 |

| **Sample input 2** | **Sample output 2** |
|---|---|
| 10 1 100<br>1 2 3 4 5 6 7 8 9 10<br>2 1 11<br>999983 100003<br>57357831<br>11<br>1 2 3 4 5 6 7 8 9 10 11 | 11<br>1023060806 |

| **Sample input 3** | **Sample output 3** |
|---|---|
| 5 1 1<br>1 2 3 2 1<br>1 1 3<br>0 0<br>1<br>2<br>1 2 | 6<br>7 |

## Note

**Explanation of the first sample**

After the first $Q_1 = 4$ operations of the example 1, the colors of the sticks in the box are as follows from left to right: $[3, 4, 4]$.

The parameters of the generator are $a = 2$ and $d = 3$. The initial seed is $X_0 = 7$. The next values of $X_i$ are calculated by the expression $X_{i+1} = (2X_i + 3)\%(10^9 + 7)$. The values of $X_0, X_1, \ldots, X_7$ are $7, 17, 37, 77, 157, 317, 637, 1277$, respectively. The last $Q_2 = 3$ operations are generated as follows:

1. The first of the last $Q_2$ operations is of type $t = X_0\%4+1 = 4$, with $k = X_1\%(3+1) = 1$. The answer after this operation is 0.

2. The following operation is of type $t = X_2\%4 + 1 = 2$, with $k = X_3 + 1 = 78$ and $c = B[X_4\%4 + 1] = 1$. The answer after this operation is 3.

3. The last operation is of type $t = X_5\%4 + 1 = 2$, with $k = X_6 + 1 = 638$ and $c = B[X_7\%4 + 1] = 1$. The answer after this operation is 3.

The last integer in the output, then, should be $(1 \cdot 0) \oplus (2 \cdot 3) \oplus (3 \cdot 3) = 15$.

# Problem D – Doubled Sequence II

Let's be direct. No stories, no worries. You will be given an integer $n$ and you must determine if there is a sequence $a$ of length $2n$ such that:

- Each value from 1 to $n$ occurs exactly twice in $a$.
- For each $i = 1, \ldots, n$, the distance between both of its occurrences is exactly $i + 1$. This is, let $L_i$ and $R_i$ be the leftmost and rightmost occurrence of $i$, respectively. Then, $R_i - L_i = i + 1$.

If there is at least one sequence, print it. Otherwise, print `-1`.

## Input

The first line contains an integer $t$ ($1 \leq t \leq 10^5$) — The number of testcases.

The following $t$ lines contain an integer $n_i$ ($1 \leq n_i \leq 10^6$) — The value of $n$ of the $i$-th query.

It is guaranteed that the sum of $n_i$ doesn't exceed $2 \cdot 10^6$.

## Output

For each query, print a single line — Print `-1` if there is no sequence that holds the conditions or $2n$ integers separated by a space representing a valid sequence.

| **Sample input 1** | **Sample output 1** |
|---|---|
| 5<br>1<br>2<br>3<br>4<br>7 | -1<br>-1<br>2 3 1 2 1 3<br>2 3 4 2 1 3 1 4<br>1 5 1 6 3 7 4 5 3 2 6 4 2 7 |

# Problem E – El Juego del Calamar

It's time for the first game of the global edition of the Squid Games.

The game is played in a tower with $n$ floors numbered from 1 to $n$, each floor containing $k$ rooms numbered from 1 to $k$. There are $m$ stairs; the $i$-th stair connects room $u_i$ on floor $f_i$ with room $v_i$ on floor $f_i + 1$.

There are $2 \cdot 10^9$ players invited to play the game. At the beginning, each of the $2 \cdot 10^9$ players must choose any room on the first floor. The game lasts for $n$ turns. In turn $t$ ($1 \le t \le n$), the following actions occur in order:

1. For every room $j$ on floor $t$, if there are more than $C_{t,j}$ players, random players are eliminated until exactly $C_{t,j}$ remain in that room.
2. If $t < n$, each surviving player in a room on floor $t$ must move to a room on floor $t + 1$ using one of the available stairs connected to their current room. Players unable to move are eliminated. Moving down is not allowed.
3. If $t = n$, all surviving players are declared winners of the first game, and the game ends.

Player 456 wants to maximize the number of players who survive. For each turn $t$, compute the maximum number of players that can be alive at the end of that turn, assuming optimal choices for the initial distribution and movements. Note that for each $t$, these values are calculated independently.

## Input

The first line contains three integers $n$, $k$, and $m$ ($1 \le n \le 1000$, $1 \le k \le 15$, $0 \le m \le (n-1) \cdot k^2$).

The next $n$ lines each contain $k$ integers $C_{i,1}, C_{i,2}, \ldots, C_{i,k}$ ($0 \le C_{i,j} \le 10^8$), representing the capacity of each room.

Then $m$ lines follow. Each contains three integers $f_i$, $u_i$, and $v_i$ — describing a stair from room $u_i$ on floor $f_i$ to room $v_i$ on floor $f_i + 1$.

## Output

Print a single line with $n$ integers — the answer for each turn.

| **Sample input 1** | **Sample output 1** |
|---|---|
| 2 2 1<br>3 7<br>4 2<br>1 1 2 | 10 2 |

| **Sample input 2** | **Sample output 2** |
|---|---|
| 3 3 5<br>8 2 4<br>3 4 8<br>4 5 0<br>1 1 1<br>1 2 2<br>1 3 3<br>2 1 1<br>2 2 2 | 14 9 5 |

# Problem F – Fantastic Robot

The Fantastic Four have been defeated by the evil Doctor Doom, and are being held hostage in his castle in Latveria, and it's up to HERBIE the robot to save them. HERBIE has managed to infiltrate the castle and now just needs to reach the cell where the Fantastic Four are held to free them.

Doom's castle can be represented as 2D binary matrix of $NxM$. If a cell has a value of 1 it means there is a wall there, while a 0 means it's a free space. HERBIE can move only through free spaces, but since he's a robot he can only move in some specific ways.

HERBIE can follow one of 4 possible **instructions**. Assuming HERBIE is on position $(x, y)$ then:

- $U$ means HERBIE will try to move to $(x-1, y)$
- $D$ means HERBIE will try to move to $(x+1, y)$
- $L$ means HERBIE will try to move to $(x, y-1)$
- $R$ means HERBIE will try to move to $(x, y+1)$

If the position HERBIE is trying to move to is a wall or it's outside of the matrix, HERBIE will simply not move and continue with the next instruction.

However HERBIE can't just do any instruction he wants, he has to follow one of its many preprogrammed **orders**. An **order** is an ordered set of instructions. If HERBIE chooses to execute certain order, then he has to do all of the movements described in it and in their given order, without changing them around.

As time is running out, HERBIE wants to reach the Fantastic Four moving the least amount of time. He has asked you to help him find out the minimum amount of movements he has to do to reach the Fantastic Four and save them! To properly save them HERBIE has to finish at their cell after he completes the instructions of an order. Passing by the cell while not completing an order doesn't count as reaching them.

## Input

In the first line two numbers $N$, and $M$ $(1 \leq N, M \leq 10^2)$, the dimension of the matrix.

In the next $N$ lines, $M$ characters 1 and 0, the matrix representing the map.

In the next line 2 numbers $A_x$ and $A_y (0 \leq A_x \leq N-1, 0 \leq A_y \leq M-1)$, the place where HERBIE is currently at.

In the next line 2 numbers $B_x$ and $B_y (0 \leq B_x \leq N-1, 0 \leq B_y \leq M-1)$, the place where the Fantastic Four are being held. Is guaranteed both of these places will have a 0 in the matrix.

In the next line a number $K (1 \leq K \leq 10^4)$ the number of different **orders** HERBIE knows.

In the next $K$ lines, a string $S_i$ made up of $U$, $D$, $L$, $R$, representing the $i$-th order HERBIE knows (it is guaranteed the sum of the lengths of all the strings is less than $10^4$)

## Output

A number indicating the minimum number of movements HERBIE will need to do. If it's impossible for HERBIE to reach the Fantastic Four print -1.

| **Sample input 1** | **Sample output 1** |
|---|---|
| 2 2<br>00<br>10<br>0 0<br>1 1<br>2<br>RR<br>D | 2 |

# Problem G – Gatuno's Descent into Psychopathy

The once-kind feline *Gatuno* is a very good and heart-guided person. As every good person, his goodness can be measured. He carries a heart of size $H_1$. Unfortunately, now he seeks to shed his humanity. His twisted goal: reduce his heart to size $H_2$ or smaller through a horrifying ritual — biting innocent dogs.

Each bite comes at a terrible cost to his remaining conscience, but makes subsequent bites easier in his dark transformation...

After biting $n$ dogs, the rule of shrinking heart is:

- His heart shrinks: $H_n = H_1 \times \left(\frac{B-1}{B}\right)^n$ Therefore, each subsequent bite requires $\frac{1}{B}$ of the previous emotional pain.

Help Gatuno to find the minimum **minimum number of dogs** Gatuno must bite to reduce his heart size to $H_2$ or smaller.

## Input

The input consists of multiple test cases.

The first line contains an integer $T$ ($1 \leq T \leq 10^5$).

Each of the next $T$ lines describes one test case with three space-separated integers:

- $H_1$: Initial heart size ($1 < H_1 \leq 10^{12}$) — Yes, Gatuno was a **really good** feline - $H_2$: Target heart size ($0 < H_2 < H_1$) - $B$: Brutality factor ($2 \leq B \leq 2 \times 10^5$)

## Output

For each test case, output a single integer — the **minimum number of dogs** Gatuno must bite to reduce his heart size to $H_2$ or smaller.

| **Sample input 1** | **Sample output 1** |
|---|---|
| 3<br>100 50 2<br>1000 1 10<br>1000 100 10 | 1<br>66<br>22 |

# Problem H – Heritage of Acatlan

In the Library of Acatlán, an ancient spell is written as a long string $S$ of length $N$. The spell also contains a secret incantation $T$, a string of length $M$.

The power of the spell is measured by the number of subsequences of $S$ that are equal to $T$.

The oracle of Acatlán allows you to perform at most one modification on $S$: you may change a single character in $S$ to any **other** lowercase English letter.

Your task is to count the power of all the possible spells with at most one modification.

**Definition:** A string $U$ is called a subsequence of a string $V$ if $U$ can be obtained from $V$ by deleting zero or more characters without changing the relative order of the remaining characters.

## Input

The first line contains two integers $N$ and $M$ ($1 \leq N \leq 10^5$, $1 \leq M \leq 60$) — the lengths of the strings $S$ and $T$.

The second line contains the string $S$.

The third line contains the string $T$.

Both strings consist of lowercase English letters.

## Output

Print a single integer, the accumulated power of all possible spells with at most one modification number of subsequences of $S$ that contain $T$ after at most one modification, because this number can be very big, print it modulo $10^9 + 7$

| **Sample input 1** | **Sample output 1** |
|---|---|
| 3 2<br>cac<br>ac | 27 |

| **Sample input 2** | **Sample output 2** |
|---|---|
| 3 3<br>abc<br>abc | 1 |

## Note

For the first sample case, if we modify any of the last two characters, we won't have any subsequence equal to $T$. In addition, the only modification of the first character that makes $S$ to have two subsequences equal to $T$ is with the letter '`a`'. The other modifications or no modification just keep the number of subsequences in 1, so we have $2 \times 1 + 1 \times 25 = 27$.

For the second sample case, the only scenario in which the string $S$ contains a subsequence equal to $T$ is when we don't apply modifications at all. Any modification would imply that $S$ won't contain $T$ as a subsequence. Therefore, the answer is 1.

# Problem I – Isaac and MOD Convolution

Isaac is a passionate competitive programmer, especially fascinated by convolution techniques used in ICPC problems. During the lectures of last year's *Training Camp Mexico* (TCMX for short), he mastered *bitwise* convolutions:

- **AND convolution**: for each $k$,

$$C[k] = \sum_{i \,\&\, j = k} A[i] \cdot B[j]$$

- **OR convolution**: for each $k$,

$$C[k] = \sum_{i \mid j = k} A[i] \cdot B[j]$$

- **XOR convolution**: for each $k$,

$$C[k] = \sum_{i \oplus j = k} A[i] \cdot B[j]$$

He implemented all of them successfully with SOS DP and used them to solve problems involving fast subset transforms.

This year, during the *Tercera Fecha del Gran Premio de México ICPC 2025*, which coincides with the third contest of this year's TCMX, Isaac is facing a problem involving a new and intriguing type: MOD convolution. It is defined similarly to the bitwise convolutions, but replacing the bitwise index condition with a modular remainder condition:

$$C[k] = \sum_{\substack{1 \le i \le n \\ k < j \le m \\ i \equiv k \bmod j}} A[i] \cdot B[j]$$

As everyone knows, MOD convolution can be easily calculated using ■■■.

The problem asks, given arrays $A$ of length $n$ and $B$ of length $m$, to compute:

$$\sum_{k=0}^{\infty} k \cdot C[k].$$

Where $C[k]$ is defined as in the MOD convolution above.

Just like Isaac, you are also competing in the *Tercera Fecha del Gran Premio de México ICPC 2025*, and you should find the answer for this problem.

## Input

The first line contains two integers $n$ and $m$ ($1 \le n, m \le 10^6$).

The second line contains $n$ integers $A_1, \ldots, A_n$ ($0 \le A_i \le 10^6$).

The third line contains $m$ integers $B_1, \ldots, B_m$ ($0 \le B_j \le 10^6$).

## Output

Print a single integer — the answer to the problem.

| **Sample input 1** | **Sample output 1** |
|---|---|
| 2 2<br>5 3<br>6 4 | 20 |

| **Sample input 2** | **Sample output 2** |
|---|---|
| 3 4<br>3 4 8<br>2 0 2 5 | 197 |

# Problem J – Juan vs Frank

In ancient times, when the gods still contested the fate of mankind, two immortal brothers clashed in a sacred ritual: Juan, Lord of Chaos, and Frank, Guardian of Order. Before them stood $n$ ancestral columns, each filled with fragments of divine power.

The brothers played these duels to see who would rule over the mortal realm.

Each duel was foretold by a prophecy, specifying the columns from $l$ to $r$. Within this range, the brothers engaged in their duel under unbreakable laws:

- Frank always starts the duel.
- The players alternate turns.
- On their turn, a player had to choose one column and remove any positive number of fragments from it.
- The one who faced an empty field, with no fragments left to remove, was defeated.
- Both players play optimally.

The high priests, gifted with the power of foresight, have recorded $Q$ events. Each event is either:

- A prophecy, foretelling a duel over a range of columns $(l, r)$.
- A ritual, adding $x$ fragments to column $k$, altering its power.

Now, the temple calls upon you to ensure the chronicles of Order and Chaos remain true

## Input

The first line contains two integers $N$ and $Q$ ($1 \leq N, Q \leq 10^6$) — the number of columns and the number of events.

The second line contains $N$ integers $B_1, B_2, \ldots, B_N$ ($1 \leq B_i \leq 10^9$) — the initial values of the columns.

Each of the next $Q$ lines contains a single event in one of the following formats:

- P l r the character `P` denotes a **prophecy** on the range $[l, r]$ ($1 \leq l \leq r \leq N$).
- R k x the character `R` denotes a **ritual**, adding the integer $x$ to the $k$-th column ($1 \leq k \leq N, -10^9 \leq x \leq 10^9$).

It is guaranteed that all inputs are valid and follow the above formats.

## Output

For each prophecy, print "JUAN" if Juan is predicted to win, or "FRANK" if Frank is predicted to win.

| **Sample input 1** | **Sample output 1** |
|---|---|
| `10 7`<br>`1 2 3 4 5 6 7 8 9 10`<br>`P 1 2`<br>`P 1 3`<br>`P 1 4`<br>`R 3 1`<br>`P 1 5`<br>`P 1 6`<br>`P 1 7` | `FRANK`<br>`JUAN`<br>`FRANK`<br>`FRANK`<br>`JUAN`<br>`FRANK` |

# Problem K – Kilometric Intersection

On the Pan-American Highway, two maintenance crews work on different road segments. Each segment is a closed interval on a line (in kilometers). To coordinate resources, we only need the *length* of the overlap between both segments.

Given two closed intervals $[a, b]$ and $[c, d]$ (with $a \leq b$ and $c \leq d$), print the length of their intersection on the real line. If they do not overlap (or only touch at a single point), the length must be 0.

The length of a closed interval $[x, y]$ is defined as $y - x$.

## Input

The first line contains an integer $T$ ($1 \leq T \leq 10^5$), the number of test cases. Each of the next $T$ lines contains four integers $a, b, c, d$ ($-10^{18} \leq a \leq b \leq 10^{18},\ -10^{18} \leq c \leq d \leq 10^{18}$).

## Output

For each test case, print a single integer: the intersection length.

| **Sample input 1** | **Sample output 1** |
|---|---|
| 6<br>3 7 1 5<br>0 0 0 0<br>-5 -1 2 8<br>1 10 3 6<br>-1 4 4 10<br>-10 -2 -7 0 | 2<br>0<br>0<br>3<br>0<br>5 |

## Note

- If intervals only touch at an endpoint (e.g., $[0, 1]$ and $[1, 2]$), the intersection is $\{1\}$, whose length is 0.
- Use 64-bit integers to avoid overflow.

# Problem L – Los Ratones III

In the distant hills, there was the city of Ratonia, home of the Ratones. Its streets were connected in a very special way: between any two houses, there was exactly one unique path.

Each house in Ratonia had a unique identifier (from 1 to N). Additionally, every street connecting two houses had its own value (from 1 to 1000000000).

After returning from a tournament, the Ratones received Q letters from their fans.

In every letter, the fans asked the same request:

"Dear Ratones, could you tell us the sum of all edge values along the path that connects the house with identifier u and the house with identifier v after multiplying the hole path by x?"

Since the Ratones don't know how to answer these requests themselves, they have asked for your help.

## Input

The first line of input contains two integers $N$ and $Q$ ($1 \leq N, Q \leq 100000$), the number of houses and the number of letters.

The next $N-1$ lines contains tree integers $u$ $v$ $c$ ($1 \leq u, v \leq N$) ($u! = v$) ($1 \leq c \leq 1000000000$), this means that the there exists a path between u and v with value c

The next $Q$ lines contains two integers $u$ $v$ $x$ ($1 \leq u, v \leq N$) ($1 \leq x \leq 1000000000$), the question that the fans asked

## Output

For each letter respond to the request. Since the answer may be very large, output it modulo 1000000007.

| **Sample input 1** | **Sample output 1** |
|---|---|
| 5 3<br>1 2 1<br>2 3 1<br>3 4 1<br>4 5 1<br>1 2 2<br>1 5 1<br>1 5 2 | 2<br>5<br>10 |